# The Great Stock-Out Mystery

## An E-commerce Demand-Supply System built on Apache Hadoop




Ideas, architecture, working MapReduce code, real results, and a roadmap.

Told as a story, drawn as pictures, shown with code you can run today.

November 2026 | All data in this pack is synthetic, generated for the demo

# 1 Story time: the night the kurta sold out

It is the first night of the festive sale at **KartKing**, an imaginary online store with three warehouses: Kolkata (**KOL**), Delhi (**DEL**) and Bengaluru (**BLR**).

At 9:40 pm the best-selling blue kurta sells out in Delhi. Customers see **Out of stock** and leave for a competitor. At the same moment the Kolkata warehouse is sitting on a month of the very same kurta. Nobody noticed, because the **demand** numbers lived in the orders database, the **stock** numbers in the warehouse system, and the **supplier delivery times** in somebody's spreadsheet.

Priya, the junior data engineer, is asked one question: *"Can we see this coming?"* This pack is Priya's answer, and it is yours to build on.

**The mystery to solve**

For every product in every warehouse, will we run out **before the next delivery can arrive**? If yes, how many units should we order today? Hadoop matters because a real store produces millions of order lines a day, far too many for one laptop to chew through.

# 2 The problem in one breath

A **demand-supply system** watches two things and the gap between them:

- **Demand**: how many units customers are likely to buy per day (we forecast it from order history).
- **Supply**: what is on the shelf now, how long the supplier takes to deliver (**lead time**), and how reliably they ship what we ask for (**fill rate**).
- **The gap**: days of stock left versus days until a new delivery lands. When stock lasts fewer days than the lead time, we are going to stock out.

| Quantity | How it is calculated in the demo | Plain-English meaning |
|---|---|---|
| Forecast / day | average of the last 7 days x trend (last 7 vs previous 7, kept between 0.8 and 1.6) | How fast we expect it to sell next week |
| Cover (days) | on_hand / forecast per day | How many days the shelf will last |
| Reorder qty | max(0, ceil(forecast x lead_time x 1.25 / fill_rate - on_hand)) | How much to order now so we survive the wait |
| Risk label | CRITICAL if cover < lead time; WATCH if cover < 1.5 x lead; OVERSTOCK if cover > 45 days; else OK | A traffic light for the planner |

**Why divide by the fill rate?**

If a supplier usually ships only 90% of what you order, you must order about 1/0.9 of what you need. The 1.25 is a safety cushion for demand surprises. Both are knobs you can tune and defend in a viva.

## 3 Why Hadoop? The exam-hall story

Imagine 10 million exam answer sheets. One teacher needs years. Hadoop hires 100 teachers and runs the hall for them:

- **HDFS** is the filing room. It cuts the pile into stacks (128 MB blocks by default) and keeps 3 copies of each, so a fire in one cupboard loses nothing.
- **YARN** is the principal. It decides which teacher marks which stack, and sends the work to the teacher who already holds the stack.
- **MapReduce** is the marking scheme. *Map*: each teacher marks their own stack. *Shuffle and sort*: marks are regrouped by student. *Reduce*: one teacher adds up each student's total.

| Tool | Job in this project | Memory hook |
|---|---|---|
| HDFS | Stores raw and curated order, inventory and supplier files | The filing room |
| YARN | Schedules every MapReduce / Hive / Spark job on the cluster | The principal |
| MapReduce | Daily demand sums, the demand-vs-stock join, weekly trends | The marking scheme |
| Hive | SQL over HDFS files: marts, uplift reports, planner queries | SQL on a giant shelf |
| HBase | Millisecond look-up of one SKU's risk for a dashboard or app | The pigeonhole desk |
| Sqoop / Flume / Kafka | Bring orders in from MySQL (batch) and clickstream (live) | The delivery vans |
| Oozie / Airflow | Runs the jobs nightly, in the right order | The alarm clock |
| Spark (optional) | Faster iterative jobs and ML forecasts on the same cluster | The sports car |

> **Honest note**
>
> Many companies now use Spark instead of classic MapReduce because it is faster. Your assignment says Apache Hadoop, so the core jobs here are MapReduce on HDFS and YARN with Hive on top. Spark appears only as an optional upgrade, and the demo does not need it.

# 4 Eight project ideas (pick your adventure)

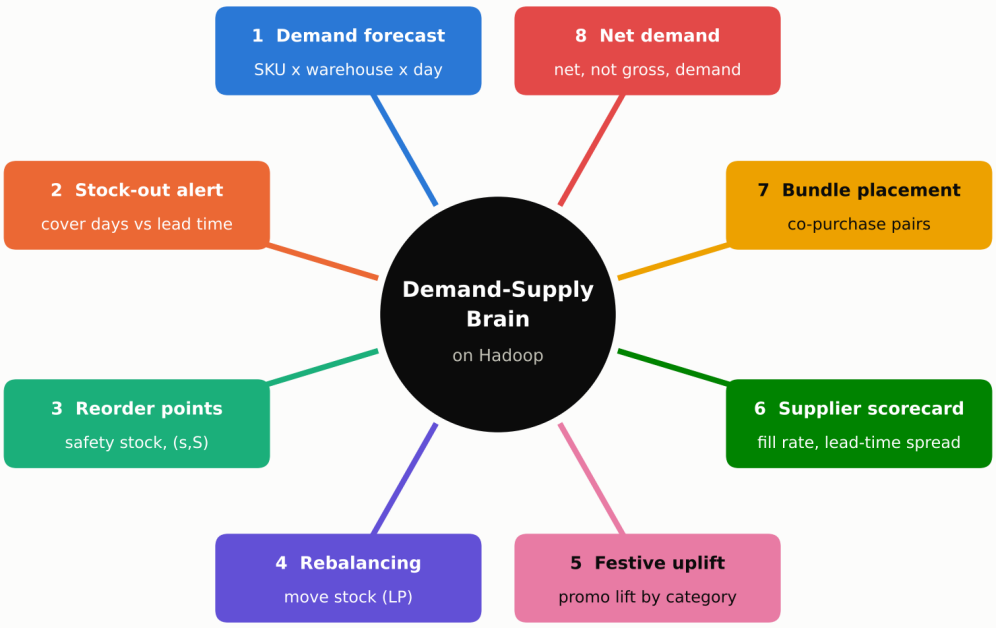



Figure 1. Eight ideas that all hang off one data pipeline. Pick one headline idea and two supporting ones.

| # | Idea | Question it answers | Hadoop pieces | Effort |
|---|---|---|---|---|
| 1 | Demand forecasting | How many units of each SKU will each warehouse sell next week? | MapReduce aggregates, Hive features, optional Spark MLlib | Medium |
| 2 | Stock-out early warning | Which SKU-warehouse pairs run dry before restock arrives? | MapReduce join (built in the demo) | Low |
| 3 | Smart reorder points | How much to order, and when, given lead time and fill rate? | Reducer maths; later (s,S) policy or newsvendor | Low to medium |
| 4 | Warehouse rebalancing | Which surplus stock should move to which short warehouse? | Hive for inputs, small LP solved outside Hadoop | Medium to high |
| 5 | Festive-sale uplift | How much does a sale lift each category, and for how long? | Hive group-by on week and category | Low |
| 6 | Supplier scorecard | Which suppliers are late or short-shipping most often? | MapReduce on PO and delivery logs | Low |
| 7 | Bundle pre-positioning | Which items are bought together, so we stock them together? | MapReduce pair counting (co-occurrence) | Medium |
| 8 | Returns-adjusted demand | What is net demand after returns and cancellations? | Join orders with returns in MapReduce or Hive | Low |

### My recommendation

Build **ideas 2 + 3 + 1** as one pipeline and call it the **Early-Warning Reorder Engine**. Add **idea 5** as the wow slide, because the festive spike makes the story obvious. Ideas 2 and 3 are already working in the demo; idea 1 is the natural next step because the demo's forecast is deliberately simple.

## 5 The architecture: five stops on one journey

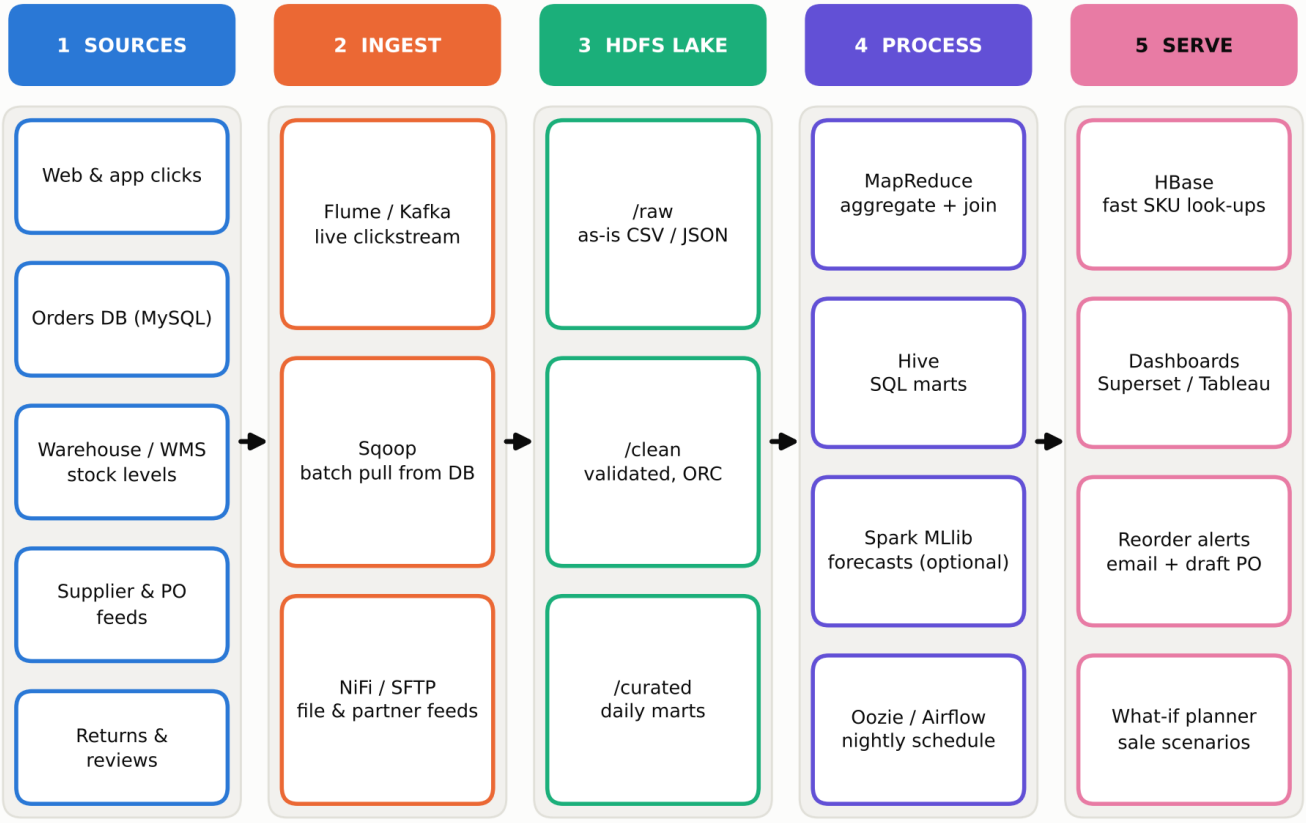



Figure 2. Orders, stock and supplier data travel left to right, from sources to decisions. Everything in column 4 runs on YARN and reads and writes HDFS.

- **Sources**: clicks, orders, warehouse stock, supplier feeds, returns.
- **Ingest**: Sqoop copies tables from MySQL in batches; Flume or Kafka carries live events; NiFi or SFTP handles partner files.
- **HDFS lake**: */raw* keeps data exactly as received, */clean* holds validated columnar files, */curated* holds the small daily tables the business reads.
- **Process**: MapReduce does the heavy lifting, Hive gives SQL access, Oozie or Airflow runs it nightly. Spark MLlib is optional.
- **Serve**: HBase for instant look-ups, dashboards for planners, alerts that draft a purchase order.

## 6 The data: two tables are enough to start

| File | Columns | One row means |
|---|---|---|
| orders.csv | order_id, date, sku, category, warehouse, qty, price | One order line: a customer bought qty units of a SKU, shipped from one warehouse |
| inventory.csv | sku, warehouse, on_hand, lead_time_days, supplier_fill_rate | Today's stock of one SKU in one warehouse plus how that supplier behaves |

The generator writes **39,859** order lines for 40 SKUs across 3 warehouses over 100 days (1 Aug to 8 Nov 2026). It plants real-world patterns on purpose: weekend bumps, a slow upward trend, and a **festive-sale jump starting 25 October**. That gives the forecast something honest to chase.

When you move to real data, good public starting points are the **Olist Brazilian E-Commerce** dataset, the **M5 Forecasting (Walmart)** dataset and the **Instacart Market Basket** dataset, all on Kaggle. Check each licence, and

note that none of them ships with warehouse stock or lead times, so you will need to simulate or assume those columns.

## 7 Three MapReduce jobs, explained as a relay race

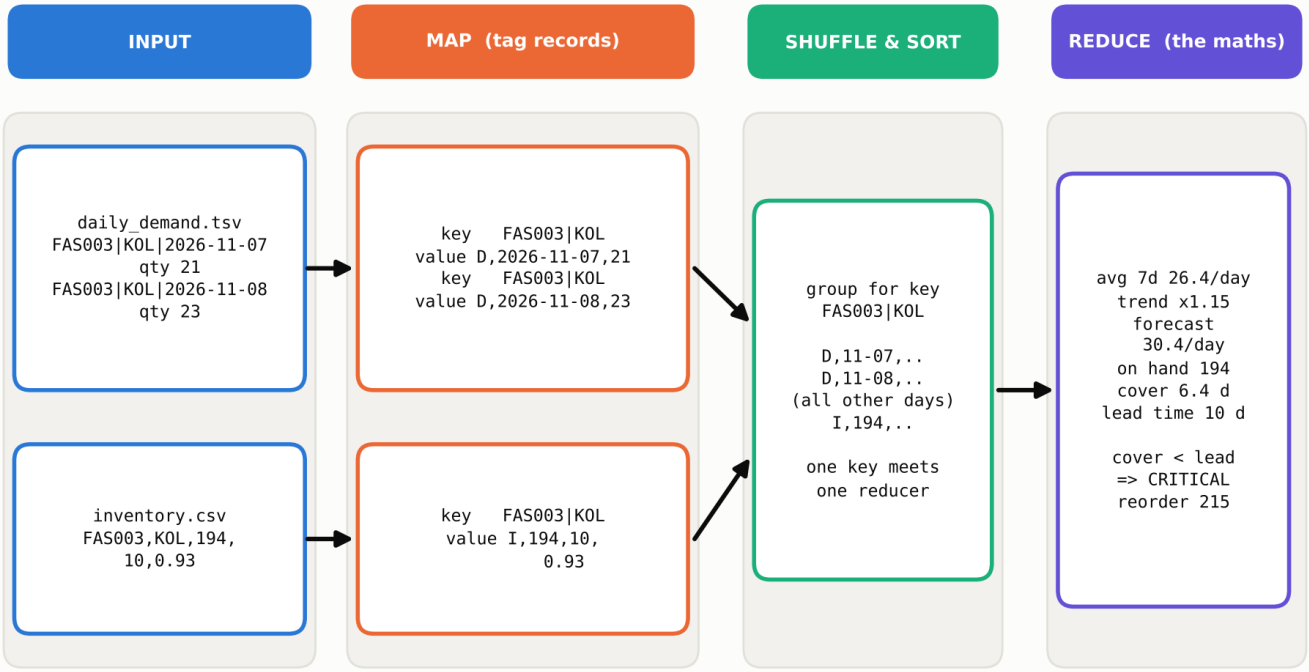



Figure 3. Job 2 follows one real key (SKU FAS003 at the Kolkata warehouse) through map, shuffle and reduce. Hadoop moves the data; your code only writes map() and reduce().

| Job | Map emits | Reduce does | Output |
|---|---|---|---|
| 1 Daily demand | (sku\|warehouse\|date, qty) for every order line | Sums quantities per key. The same code is reused as a **combiner** to shrink network traffic | 9,967 daily totals |
| 2 Supply-demand gap | (sku\|warehouse, tagged record): D for a demand day, I for the inventory row | Reduce-side **join**, then forecast, cover, risk, reorder quantity | 120 rows, one per pair |
| 3 Weekly category trend | (category\|week, qty) | Sums per key | 56 weekly totals for the trend chart |

### Four gotchas that catch everyone once

(1) Streaming sorts keys as **text**, so ISO dates like 2026-11-08 sort correctly but 8-11-2026 would not. (2) Every mapper must skip the CSV header line. (3) A reducer sees one key at a time, so keep its memory small. (4) A combiner is only safe when the operation is associative, like a sum; a plain average is not.

## 8 The code (black blocks, colourful text)

Everything below runs as ordinary **Hadoop Streaming** scripts: each reads lines from standard input and writes tab-separated key and value lines. Hadoop Streaming then supplies the shuffle and sort between map and reduce, and the demo reproduces that locally with `sort`.

**mapper_daily.py (Job 1 map)**

```
#!/usr/bin/env python3
"""Job 1 mapper: order line -> ((sku,wh,date), qty)."""
import sys
for line in sys.stdin:
    p = line.rstrip("\n").split(",")
    if p[0] == "order_id":          # skip header
        continue
    _, date, sku, cat, wh, qty, price = p
    print(f"{sku}|{wh}|{date}\t{qty}")
```

**reducer_sum.py (Job 1 reduce and combiner, Job 3 reduce)**

```
#!/usr/bin/env python3
"""Job 1 reducer (also usable as combiner): sum quantities per key."""
import sys
cur, total = None, 0
for line in sys.stdin:
    k, v = line.rstrip("\n").split("\t")
    if k != cur:
        if cur is not None:
            print(f"{cur}\t{total}")
        cur, total = k, 0
    total += int(v)
if cur is not None:
    print(f"{cur}\t{total}")
```

**mapper_gap.py (Job 2 map: tag demand D and inventory I)**

```
#!/usr/bin/env python3
"""Job 2 mapper: reduce-side join of daily demand (Job 1 output) and inventory.
Daily demand line : sku|wh|date <TAB> qty
Inventory line    : sku,warehouse,on_hand,lead_time,fill_rate
Emits key=sku|wh, value=tagged record."""
import sys
for line in sys.stdin:
    line = line.rstrip("\n")
    if "\t" in line:                            # demand record
        k, qty = line.split("\t")
        sku, wh, date = k.split("|")
        print(f"{sku}|{wh}\tD,{date},{qty}")
    elif line.startswith("sku,"):               # inventory header
        continue
    else:                                       # inventory record
        sku, wh, on_hand, lead, fill = line.split(",")
        print(f"{sku}|{wh}\tI,{on_hand},{lead},{fill}")
```

**reducer_gap.py (Job 2 reduce: forecast, cover, risk, reorder)**

```
#!/usr/bin/env python3
"""Job 2 reducer: per (sku,wh) compute forecast demand, cover, risk, reorder qty."""
import sys, math

SAFETY = 1.25           # safety factor on lead-time demand

def flush(key, demand, inv):
    if not key or not inv or not demand:
        return
    on_hand, lead, fill = inv
    days = sorted(demand)                       # ISO dates sort lexically
    last7 = [demand[d] for d in days[-7:]]
    prev7 = [demand[d] for d in days[-14:-7]] or last7
    avg7, avgp = sum(last7) / len(last7), sum(prev7) / len(prev7)
    growth = (avg7 / avgp) if avgp else 1.0
    forecast = avg7 * min(max(growth, 0.8), 1.6)        # damped trend forecast / day
```

```
    cover = on_hand / forecast if forecast else 999
    need = forecast * lead * SAFETY / fill              # demand over lead time, buffered
    reorder = max(0, math.ceil(need - on_hand))
    if cover < lead:
        risk = "CRITICAL"          # stock runs out before a new order can arrive
    elif cover < lead * 1.5:
        risk = "WATCH"
    elif cover > 45:
        risk = "OVERSTOCK"
    else:
        risk = "OK"
    sku, wh = key.split("|")
    row = [sku, wh, on_hand, f"{avg7:.1f}", f"{growth:.2f}", f"{forecast:.1f}",
           f"{cover:.1f}", lead, reorder, risk]
    print(",".join(str(x) for x in row))

cur, demand, inv = None, {}, None
for line in sys.stdin:
    k, v = line.rstrip("\n").split("\t")
    if k != cur:
        flush(cur, demand, inv)
        cur, demand, inv = k, {}, None
    p = v.split(",")
    if p[0] == "D":
        demand[p[1]] = demand.get(p[1], 0) + int(p[2])
    else:
        inv = (int(p[1]), int(p[2]), float(p[3]))
flush(cur, demand, inv)
```

**run_local.sh (the whole pipeline on one machine)**

```
#!/usr/bin/env bash
# Runs the exact Hadoop Streaming data-flow locally:  cat | map | sort | reduce
set -e
cd "$(dirname "$0")"
python3 gen_data.py

echo "== Job 1: daily demand per SKU x warehouse"
cat data/orders.csv | python3 mapper_daily.py | sort \
  | python3 reducer_sum.py > out/daily_demand.tsv

echo "== Job 2: reduce-side join -> supply/demand gap"
cat out/daily_demand.tsv data/inventory.csv | python3 mapper_gap.py | sort \
  | python3 reducer_gap.py > out/gap_raw.csv
( echo "sku,warehouse,on_hand,avg7,growth,forecast,cover_days,lead_days,reorder_qty,risk"
  sort out/gap_raw.csv ) > out/gap_report.csv

echo "== Job 3: weekly units per category"
cat data/orders.csv | python3 mapper_cat.py | sort \
  | python3 reducer_sum.py > out/weekly_category.tsv
```

**On a real Hadoop cluster (3.x)**

```
# 1. put the raw files on HDFS
hdfs dfs -mkdir -p /ecom/raw/orders /ecom/raw/inventory
hdfs dfs -put data/orders.csv    /ecom/raw/orders/
hdfs dfs -put data/inventory.csv /ecom/raw/inventory/

STREAM=$HADOOP_HOME/share/hadoop/tools/lib/hadoop-streaming-*.jar

# 2. Job 1: daily demand (combiner = reducer, because a sum is associative)
hadoop jar $STREAM \
  -files mapper_daily.py,reducer_sum.py \
  -mapper "python3 mapper_daily.py" -combiner "python3 reducer_sum.py" \
  -reducer "python3 reducer_sum.py" \
  -input /ecom/raw/orders -output /ecom/curated/daily_demand

# 3. Job 2: join demand + inventory (two -input paths feed the same mapper)
hadoop jar $STREAM \
  -files mapper_gap.py,reducer_gap.py \
  -mapper "python3 mapper_gap.py" -reducer "python3 reducer_gap.py" \
  -input /ecom/curated/daily_demand -input /ecom/raw/inventory \
  -output /ecom/curated/gap_report

# 4. look at the answer
hdfs dfs -cat /ecom/curated/gap_report/part-* | head
```

Hive: the same data seen through SQL

```
CREATE EXTERNAL TABLE orders (
  order_id BIGINT, order_date STRING, sku STRING,
  category STRING, warehouse STRING, qty INT, price DOUBLE)
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/ecom/raw/orders'
TBLPROPERTIES ('skip.header.line.count'='1');

-- Idea 5: weekly units per category, to see the festive-sale lift
SELECT category, weekofyear(order_date) AS wk, SUM(qty) AS units
FROM orders
GROUP BY category, weekofyear(order_date)
ORDER BY category, wk;
```

## 9 What the demo found

Running the three jobs on the synthetic data took under a second on one laptop. The festive sale shows up clearly: Fashion units per week rose from **1,485** in week 42 to **3,291** in week 45, about **2.2x**, and every other category jumped too.

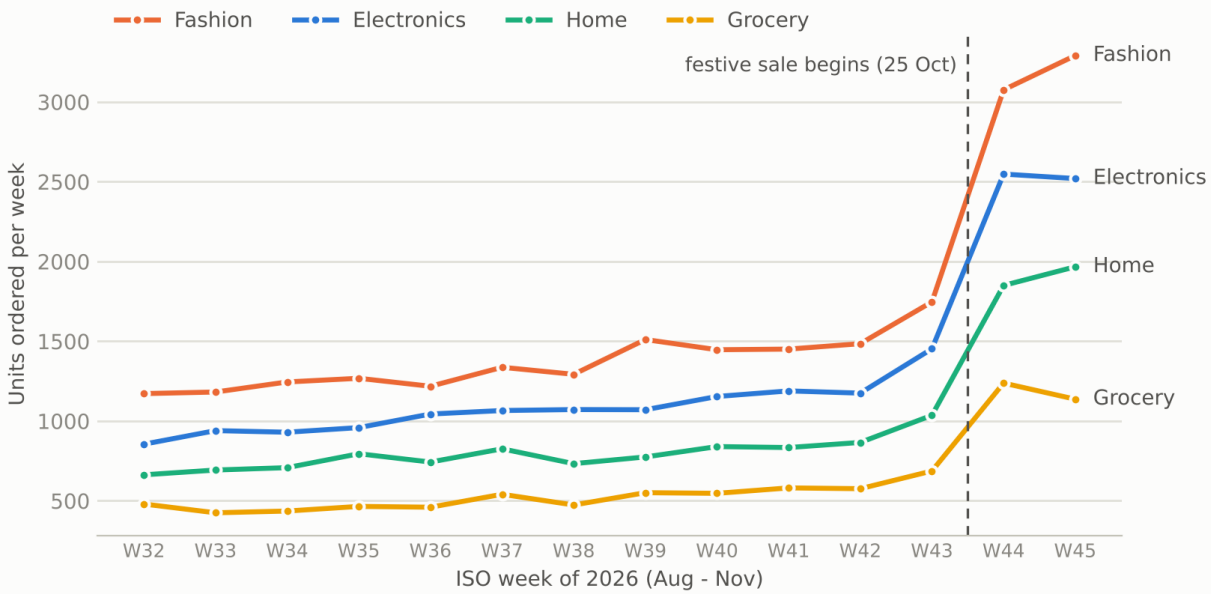



Figure 4. Weekly units by category. The dashed line marks the festive-sale start on 25 October.

Of **120** SKU-warehouse pairs, **33** are CRITICAL (stock ends before delivery can arrive), **19** are on WATCH, **62** are OK and **6** are overstocked. By warehouse, CRITICAL pairs are Kolkata 15, Delhi 10 and Bengaluru 8. The engine suggests ordering **2,621** units across the critical pairs.

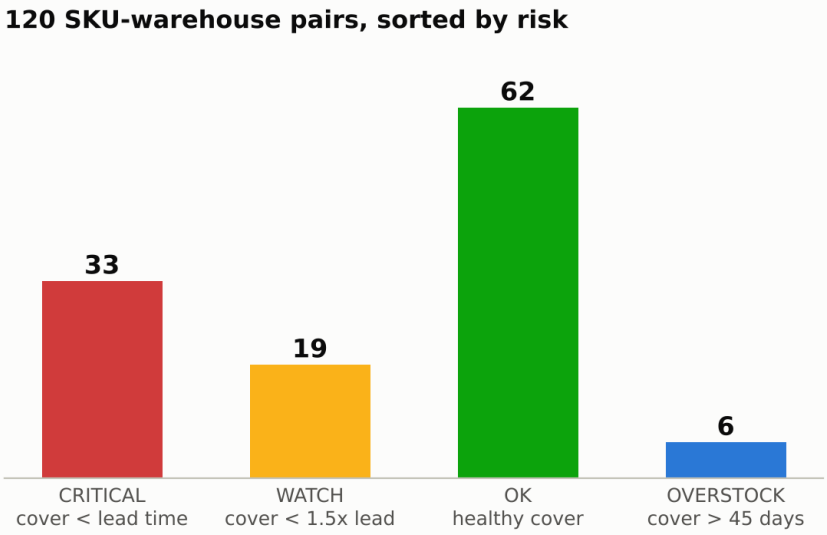



Figure 5. How many SKU-warehouse pairs fall in each risk band.

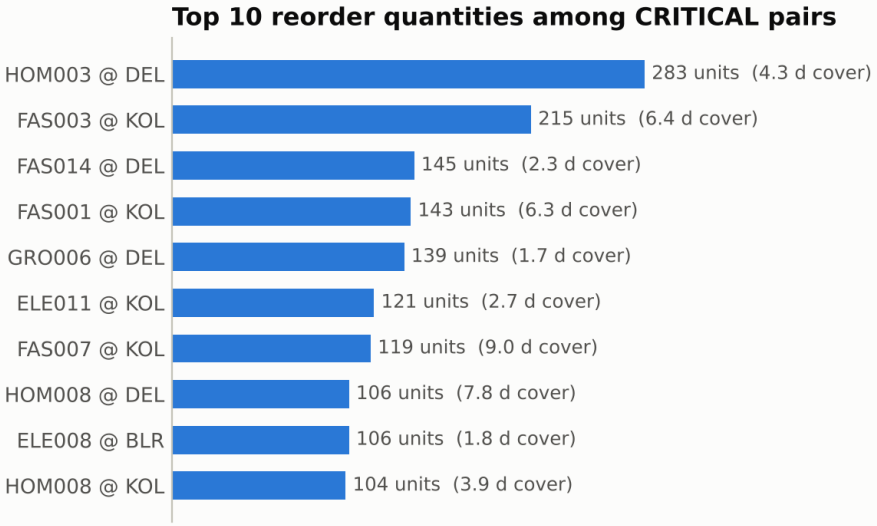



Figure 6. The biggest orders the engine would place, with the days of cover each pair has left.

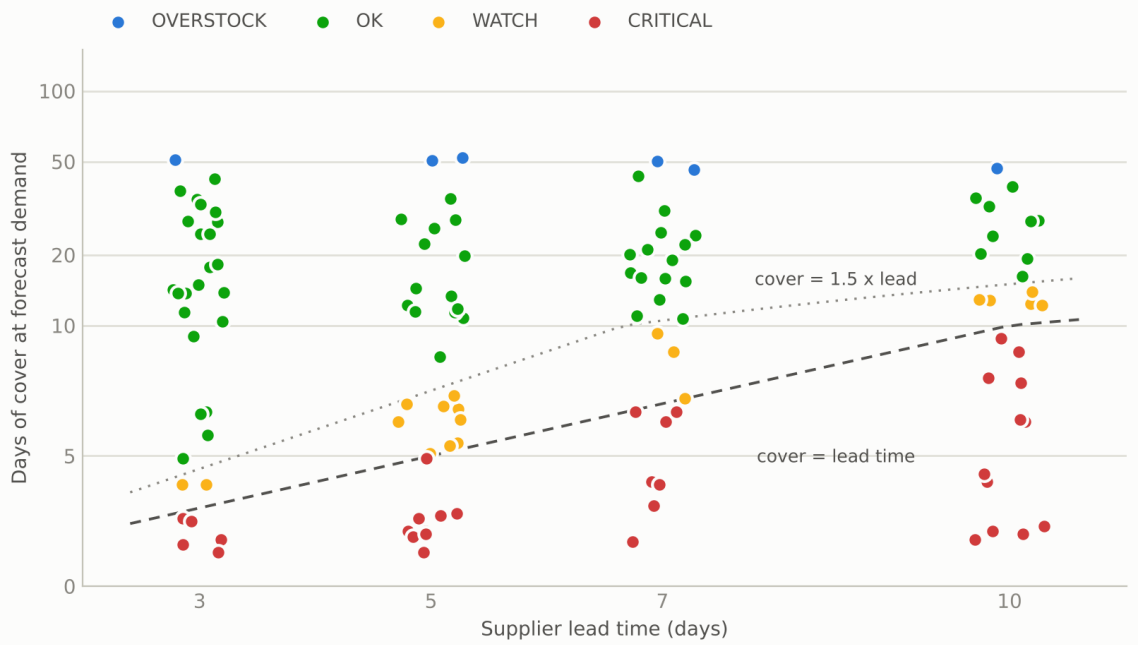



Figure 7. Each dot is one SKU-warehouse pair. Anything below the dashed line will stock out before the supplier delivers.

**Be honest about what this proves**

The data is synthetic, so these results show that the **pipeline works**, not that the forecast is accurate for any real store. The forecast is a plain 7-day average with a damped trend. With real data, hold out the last 14 days, forecast them, and score the forecast with **WAPE** (total absolute error divided by total actual units) before trusting any reorder quantity.

## 10 Your roadmap and submission checklist

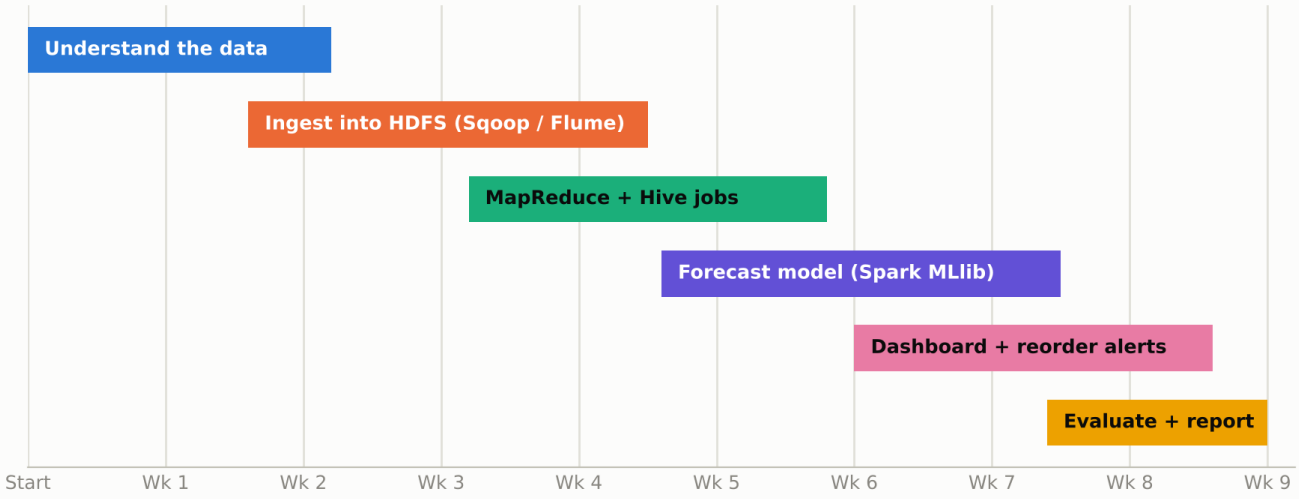



Figure 8. A nine-week plan. Shrink or stretch it to fit your deadline; the order of the steps stays the same.

- **Report skeleton**: problem and motivation, architecture, data description, job design, results, limitations, future work.
- **Demo script**: show raw files on HDFS, run Job 1 and Job 2, open the gap report, then show the dashboard.
- **Evidence**: screenshots of the YARN job page, HDFS file listing, and the risk chart.
- **Honesty slide**: what the data is, what the forecast cannot do, and how you would validate it.

### Likely viva questions, with short answers

| Question | Short answer |
|---|---|
| Why a combiner? | It pre-sums on each mapper, so far less data crosses the network in the shuffle. It is safe here because a sum is associative. |
| Why a reduce-side join? | Both inputs can be large and it works with plain Streaming. Because inventory is small, a map-side join using the distributed cache would be faster; that is a good upgrade. |
| What if one SKU is hugely popular (skew)? | One reducer gets overloaded. Use a combiner, add a random salt to the key, or split the key and re-merge. |
| Why Hive and HBase both? | Hive is for batch SQL over many rows; HBase is for low-latency single-row look-ups from an app. |
| How do you know the forecast is good? | Backtest on held-out days and report WAPE or MAPE against a naive baseline such as last week's demand. |
| Block size and replication? | Defaults in Hadoop 3 are 128 MB blocks and 3 replicas; both are configurable per file. |

## 11 Level-ups and where to go next

- **Better forecast**: add day-of-week and festival features, then try gradient boosting or ARIMA-style models; train them with Spark MLlib on the curated tables.
- **Smarter reorder policy**: replace the fixed 1.25 with a safety stock from demand variability and lead-time variability, or use an (s,S) policy.
- **Rebalancing**: a small linear program can decide how many units to ship between warehouses; solve it outside Hadoop with a solver and read the inputs from Hive.
- **Live view**: stream order events with Kafka, keep today's counts in HBase, and refresh the risk label every few minutes.
- **Scale test**: generate 50 to 100 times more rows and compare one-reducer against many-reducer run times for a clean performance graph.

> **Make it yours: three quick questions**
>
> 1) Is this a course project, an internship task or a research paper, so I can match the depth and the report format? 2) Do you have a real dataset, or should I stay synthetic? 3) Do you have access to a Hadoop cluster, or should I write steps for a single-node setup on Linux, WSL or Docker? Answer any of these and I will tune the pack.